

Migrating from a legacy engine to MessageFoundry

Early Access · MessageFoundry v0.3.2 · July 2026

Moving live clinical interfaces off an established engine is a careful, reversible exercise — not a big-bang rewrite. This guide is written for the interface analyst and integration developer who already runs feeds on Corepoint, Mirth, Cloverleaf, Rhapsody, or Ensemble and wants a clear, low-risk path to MessageFoundry.

It walks through the four phases of a real migration: **inventory** what you have, **map** the incumbent's concepts onto MessageFoundry's building blocks, **test and validate** in parallel against real-shaped traffic, and **cut over a feed at a time with a one-step rollback**.

Corepoint, Mirth Connect, Cloverleaf, Rhapsody, and Ensemble are trademarks of their respective owners. MessageFoundry is an independent project and is not affiliated with, sponsored by, or endorsed by any of those vendors. Nothing here grades another vendor's product: the comparisons are **concept translations** meant to help you reuse vocabulary you already know, not claims about what any incumbent can or cannot do.

Why teams migrate

Most teams we talk to are not unhappy with what their incumbent engine *does* — they are unhappy with how it is *owned*. In the low-code and embedded-scripting engines alike, the interface logic lives inside the vendor's editor, runtime, or database, which makes it harder to diff, review, and reproduce across environments than an ordinary source file.

MessageFoundry takes a different posture: **your configuration is code you own and version-control**. Connections, routing, and transforms live in an org-owned git repository, not in the engine's data store. Every change is a reviewable diff with an author and full history; rolling back is a `git` operation; and the same modules run unchanged in dev, staging, and production, with only per-environment values differing. `messagefoundry init` scaffolds that repo for you — a starter feed, per-environment value files, a CI workflow, and a pinned engine version — and the engine itself stays a pinned dependency your team never edits. Releases are signed and ship an SBOM.

Be clear-eyed about one thing up front: **routing and transform logic is always Python**. The tooling writes a lot of that Python for you — wizards, scaffold snippets, a cookbook, and a structured Steps view that edits a real Handler file — and a connection's *transport* settings can be edited as data in a form. But the artifact on disk and the thing that executes is always a plain `.py` file. Code-first is the differentiator, not a fallback.

Before you start: the mental-model shift

The single most important thing to internalize is this: **MessageFoundry has no "channel" object.**

Where an incumbent engine gives you one *channel* that bundles a source, a filter, transformers, and destinations into a single unit, MessageFoundry models the same flow as a **graph wired by name** — a set of small, independent nodes joined by named edges, like boxes connected by arrows in a flowchart. There are four building blocks:

Building block	What it is
Connection	An endpoint that <i>receives</i> (inbound) or <i>sends</i> (outbound) messages. Seventeen connector types ship today — MLLP, raw TCP, X12, local/UNC files, SFTP and FTP/FTPS, an inbound HTTP listener, REST, SOAP, database, FHIR, DICOM and DICOMweb, SMTP and Direct, a timer, and two internal ones. Every message in or out is counted and logged.
Router	A small pure function bound to one inbound. It sees every received message and returns the name(s) of the Handler(s) to forward to. It may filter by returning nothing.
Handler	A small pure function that takes a message from a Router, filters and transforms it, and returns one or more Send s to outbound Connections.
Message store	The durable queue and persistence layer — SQLite by default, or PostgreSQL / SQL Server for production.

The edges are just strings resolved when the configuration loads: an inbound names its Router, a Router returns its Handlers' names, and a Handler **Send s** to a named outbound. Nothing holds a direct reference to anything else.

This is why a migration is mostly a **decomposition exercise**. A single incumbent channel usually becomes one inbound Connection, one Router, one or more Handlers, and one or more outbound Connections — and the shared pieces (a common destination, a reused transform) get defined **once** and referenced by name, instead of being copied into every channel that needs them.

Phase 1 — Inventory your existing interfaces

Before touching MessageFoundry, build a complete picture of what you run today. For each interface, capture:

- **Source** — the transport and direction (MLLP/TCP listener, file drop, database poll, HTTP/SOAP endpoint), the listen port or path, and the upstream partner/system.
- **Destinations** — every downstream the channel delivers to, with transport, host/port, and any per-destination filtering.
- **Message types** — the HL7 v2 trigger events (ADT, ORM, ORU, SIU, DFT, MDM, VXU, ...), or X12 / FHIR / other payloads.
- **Routing logic** — the rules that decide which messages go where (often a filter on **MSH-9**, a facility, or a sending application).

- **Transforms** — field mappings, value-set translations (code sets), segment add/remove, and any enrichment from a lookup.
- **Acknowledgement behavior** — original vs. enhanced ACK, and whether the partner expects application-level NAK semantics.
- **Security** — TLS/mTLS on the wire, credentials, and any IP allowlisting.
- **Volume and timing** — peak-hour throughput (not the daily average), whether the feed is steady or intermittent, and whether delivery must be strictly ordered.

Two exercises are worth the time while you do this. First, note for each incumbent channel how many *distinct* destinations and transforms it actually contains — channels that fan out to several downstreams, or that share a transform with other channels, are the ones that benefit most from MessageFoundry's reuse-by-name model, and you will collapse a lot of duplicated configuration. Second, flag any feed that is both strictly ordered and high-rate; ordering is a single serial lane by definition, and those are the feeds you will want to split at the source (see *Deployment guidance* below).

Phase 2 — Map the concepts

The translation is mechanical once you know the vocabulary. This table maps common incumbent terms onto MessageFoundry building blocks. Incumbent terminology varies by product and version, so treat the left column as a guide, not a contract.

Incumbent concept	MessageFoundry equivalent
Channel (Mirth) / Interface	A <i>wired path</i> : one inbound Connection → Router → Handler(s) → outbound Connection(s). There is no single bundled object — you assemble it from the four blocks.
Source connector (Mirth)	An inbound Connection (<code>inbound(...)</code>).
Destination connector (Mirth)	An outbound Connection (<code>outbound(...)</code>).
Source filter / channel filter	A Router — returns the Handler name(s) to forward to, or nothing to drop the message (recorded as <code>UNROUTED</code> , never silently lost).
Transformer / destination transformer	A Handler — filters, transforms, and <code>Send</code> s.
Destination-set routing / "send to these destinations"	A Router that fans out (<code>return ["to_a", "to_b"]</code>) or a Handler that fans out (multiple <code>Send</code> s).
"E Process" (Corepoint)	A Router — often grouped in a <code>routers_<area>.py</code> file, each listing its handler(s).
"E Child" (Corepoint)	A Handler — a shared transform defined once in <code>handlers_<partner>.py</code> and named by multiple routers.

Incumbent concept	MessageFoundry equivalent
A channel that is defined but switched off	<code>deployed=false</code> on the connection — present in config and in the graph, but not wired: no connector is built, no listener binds, and its per-environment values are never resolved (so a partner whose credentials don't exist yet is still a legal, passing config). <code>auto_start=false</code> is the lighter variant: built, but started by an operator rather than at boot.
Configuration held inside the engine	Configuration in a git repo. MessageFoundry keeps interface logic out of the data store — the database holds messages, operational state, users and audit, never your routing and transforms.
Embedded JavaScript / proprietary scripting	Ordinary Python you own, plus a built-in structured HL7 transform model (read/set by field path, iterate repetitions, add/remove segments, MSH-aware re-encode with correct escaping).

A worked example

Suppose your incumbent has a channel "ACME ADT → EHR": it listens for ADT over MLLP, drops anything that is not an ADT, and forwards to your EHR. In MessageFoundry that is one small module:

```
from messagefoundry import MLLP, Send, env, handler, inbound, outbound, router

inbound("IB_ACME_ADT", MLLP(port=2600), router="acme_adt_router")
outbound("OB_EHR_ADT", MLLP(host=env("ehr_host"), port=env("ehr_port", cast=int)))

@router("acme_adt_router")
def route(msg):
    return ["acme_adt_handler"] if msg["MSH-9.1"] == "ADT" else [] # non-ADT → UNROUTED

@handler("acme_adt_handler")
def handle(msg):
    # filter / transform here
    return Send("OB_EHR_ADT", msg)
```

Note `env(...)`: the downstream host and port differ per environment, so they are resolved from `environments/<env>.toml` at load — the same module runs unchanged in dev, staging, and prod, and a referenced value that has no value makes the engine refuse the graph rather than load a silent blank. The connection naming convention is `[TYPE]_[PARTNER]_[MESSAGE]` (for example `IB_ACME_ADT` inbound, `OB_EHR_ADT` outbound), which keeps a large interface inventory legible.

Note also the shape of the inbound: MLLP listeners take **only a port**. Which interface they bind to is a service-level, per-environment operator decision (loopback by default), not something baked into the module.

Connections can be data, not just code

If you prefer to treat endpoints the way you treat channels today — as editable settings rather than code — a connection's transport configuration (type, settings, the inbound's router binding, delivery knobs) can live in a `connections.toml` file, edited by hand, from the CLI, or through the VS Code connection editor. The *routing and transform logic* stays in Python. Either way the loader produces the same registry the engine runs, with the same validation and the same egress gating.

Eleven transports are reachable from TOML today — MLLP, TCP, HTTP, file, timer, REST, database (write and poll), SOAP, SFTP and FTP. The rest (X12, FHIR, DICOM, DICOMweb, email, Direct, and the two internal inbounds) are code-first only for now and are declared in a `.py` module. Secrets are never written inline; they are referenced from the environment.

Mapping the transport for each connector

MessageFoundry ships seventeen connector types. Check the ones your feeds actually depend on against this list before you plan a wave:

Incumbent connector	MessageFoundry connector
MLLP / LLP Listener and Sender	<code>MLLP(...)</code> — inbound listener and outbound sender, with optional TLS and mutual TLS.
TCP Listener / Sender (custom framing)	<code>Tcp(...)</code> — raw TCP with configurable delimiter framing, for X12 or other non-HL7 feeds carried opaquely.
X12 over TCP	<code>x12(...)</code> — frames by the interchange itself (ISA...IEA), with optional synchronous request/response (e.g. 270 → 271 real-time eligibility) and TA1 classification on a capturing outbound.
File Reader / Writer	<code>File(...)</code> — polls or writes a local or UNC directory, with size caps, quarantine of malformed files, atomic writes, process-in-place for read-only shares, and an optional alternate Windows share credential.
FTP / FTPS / SFTP Reader and Writer	<code>Ftp(...)</code> (stdlib; <code>tls=True</code> is FTPS with a verifying certificate check) and <code>Sftp(...)</code> (the <code>[sftp]</code> extra, host-key verification on by default) — each is both a source and a destination.
HTTP Listener	<code>Http(...)</code> — an inbound HTTP/1.1 listener a partner <code>POST</code> s a JSON / XML / SOAP-envelope / FHIR body to, answered <code>202 Accepted</code> the instant the body is durably committed.
HTTP Sender	<code>Rest(...)</code> — outbound HTTP(S) client.
Web Service Sender	<code>Soap(...)</code> — outbound SOAP, including an opt-in WS-* mode with mutual TLS, WS-Security, and WS-Addressing.
Database Reader / Writer	<code>DatabasePoll(...)</code> (inbound poll) and <code>Database(...)</code> (outbound write) — always-parameterized SQL over ODBC. SQL Server is the production preset; a generic dialect reaches any database with an OS-installed ODBC driver. No JDBC — there is no JVM.
FHIR client	<code>FHIR(...)</code> — outbound FHIR REST client (create/update/transaction/batch) with conditional create/update knobs for idempotency, and SMART Backend Services client authentication.
DICOM Listener / Sender	<code>DICOM(...)</code> — inbound C-STORE SCP and outbound C-STORE SCU / C-ECHO (the <code>[dicom]</code> extra) — plus <code>DICOMweb(...)</code> for STOW-RS. Headers and Structured Reports only, no pixel data .
SMTP Sender	<code>Email(...)</code> / <code>SMTP(...)</code> , and <code>Direct(...)</code> for Direct-Project S/MIME over SMTP.
Scheduled / clock-driven channel	<code>Timer(...)</code> — interval or cron; each tick emits an operator-configured body.

Incumbent connector	MessageFoundry connector
Channel Reader / channel-to-channel	The graph itself, plus two first-class internal inbounds: <code>Loopback()</code> (a captured synchronous reply re-ingressed with its own Router) and <code>PassThrough()</code> (a 1:N internal hop a Handler <code>Send</code> s into).

What is not there. Plan a feed that depends on one of these into a later wave, or front it with a shipped transport in the interim: S3 / cloud blob storage; a POP3/IMAP email *reader* (SMTP send ships, receive does not); JMS, IBM MQ/MSMQ, and Kafka; an inbound **FHIR server facade**; the *synchronous* SOAP-envelope reply on the inbound listener; and request authentication on the HTTP listener's own socket — until that lands, an exposed listener belongs behind a reverse proxy that terminates auth. Serial (RS-232) and ASTM lab-instrument connectivity is **declined by design**, not pending.

On the format side, the honest answer for a long tail of payloads is *code-first Python handles this*: JSON, CSV/delimited, and fixed-width are read and emitted in a Handler with the standard library, no engine change needed. HL7 v2, X12, FHIR, generic XML, and DICOM headers/SR have real modeled parse-and-validate lanes. C-CDA, NCPDP, and HL7 v3 do **not** have a model yet — those payloads ride through as opaque bytes today, which is fine for pass-through and routing but not for field-level transformation.

Two reliability facts to design around

Two engine behaviors shape how you write transforms and how downstreams should behave:

- **At-least-once delivery.** MessageFoundry never silently loses a message — every received message is persisted before it is acknowledged. The trade-off is that a delivery which was sent but whose acknowledgement is lost (the peer closes, or times out after having already processed it) is retried, so a receiver can see a duplicate. A retry re-sends the stored payload, so an HL7 duplicate carries the **same MSH-10 control ID** — a downstream keyed on `MSH-10` sees a retry of a known message, not a new clinical event. Design outbound receivers to be idempotent (a natural upsert, an idempotency key, or a message-id de-dup). FHIR's conditional create/update knobs and X12's TA1 handling exist precisely for this.
- **Routers and transforms must be pure.** A Router or Handler is message-in, message-out, with no external side effects, so a safe re-run produces identical output. There are exactly two sanctioned exceptions, both **read-only** and both available only inside a live Handler: a database lookup and a FHIR read/search, for enrichment or gating. If your incumbent transforms reach out to external systems mid-transform, plan to move that work into a destination or one of those two lookups.

Phase 3 — Test and validate in parallel

The strength of a code-first configuration is that you can validate it long before any feed is cut over.

Validate the configuration itself

Every change is gated by `messagefoundry check` — the same command your commit hooks and CI run. It validates the wiring (an unknown router, a dangling handler, a duplicate name or a port conflict is a loud error, not a silent surprise), dry-runs sample messages when you supply fixtures, and constructs every connector under

this instance's security posture so that a configuration the engine would *refuse to start* fails at commit time instead of at 2 a.m. Lint and type checks run advisory-only, so a non-developer author is never blocked by a style nit. Because the wiring is just names, mistakes surface at load time rather than in production.

`messagefoundry impact` is the companion for the reshuffling you will do a lot of during a migration: it reports every referrer of a router, handler, or connection, and can plan (and then apply) a rename that rewrites the object *and* everything that names it.

Dry-run transforms with before/after diffs

The VS Code extension includes a **Test Bench** that dry-runs `.h17` files through your Routers and Handlers and shows before/after diffs, so an analyst can confirm a transform does exactly what the incumbent did, file by file. Pair it with the bundled synthetic message generator so you are never testing against real PHI.

Probe connectivity without sending a message

Before a feed goes live, `POST /connections/{name}/test` runs a **reachability probe**: it builds a fresh connector (never the live one), honors the egress allowlist, and confirms it can reach the peer — a socket connect for MLLP/TCP/X12, a `SELECT 1` for a database, an HTTP `HEAD` for REST/SOAP, a metadata read for FHIR, a C-ECHO for DICOM, a connect-and-NOOP for email, an SFTP/FTP connect, or a writability check for a file directory — **without sending any real message**. It is audited, and it is honest about what it cannot answer: a listener, a timer, or an internal inbound reports "nothing to probe" rather than a false green, and a `401 / 403` from an HTTP endpoint is a failure, not a pass. This lets you confirm firewall rules, credentials, and TLS to every downstream ahead of cutover.

Run a true parallel comparison with the tee relay

For the highest-confidence validation, run MessageFoundry **alongside** your live incumbent on real-shaped traffic before cutting over anything. The **tee relay** is a small, standalone tool shipped with the project that sits in front of both engines:

- Repoint the upstream source (for example an EHR's outbound) at the relay. The relay **acknowledges on receipt**, then forwards the **unchanged** bytes to **both** your live incumbent (production, unchanged) and a shadow MessageFoundry instance.
- Optionally, add a duplicate outbound send in your incumbent's configuration that mirrors its outbound messages to a second relay listener, so the shadow MessageFoundry can also see the incumbent's *output* for side-by-side comparison.

Run the shadow MessageFoundry's outbound connections in **simulate (shadow) mode**: it exercises the full route-and-transform pipeline, records what it *would* have sent, and finalizes the message normally, but suppresses real egress — so it can process live-shaped traffic without double-delivering to downstreams. You then compare MessageFoundry's transformed output against your incumbent's for parity, and you do it before a single real feed has moved.

A few things to understand about the relay so you use it correctly:

- **It is a validation tool for test and synthetic data only.** It is not hardened to carry production PHI, prints a warning at every start, and should run only on a trusted test segment. Use it to gain confidence, not as a permanent production component.
- Because it always ACKs on receipt, it is the acknowledgement authority to the upstream — an *application-level* NAK from your incumbent does not propagate back. The relay logs every one instead, which is exactly the record you want during a parity run.
- It is a **fail-closed relay, not durable store-and-forward**. If the live (production) path becomes unreachable, the relay stops accepting new connections so the upstream sees the outage and queues on its side; you restart it once the production path is healthy.
- The shadow leg is decoupled, so a slow or down shadow MessageFoundry never back-pressures the production path.
- It records one metadata row per forwarding leg (outcome, ACK code, control ID, message type, size, reason) and can export that log as JSON metadata — no message bodies — for review.

Phase 4 — Stage the cutover, keep rollback one step away

Migrate **one feed at a time**, not the whole engine at once. A typical per-feed cutover:

1. **Author and review** the feed's module(s) in your config repo as an ordinary pull request, gated by `messagefoundry check`. A feed whose partner is not live yet can be committed with `deployed=false` — it stays in the graph and the documentation without binding a listener or needing credentials that don't exist yet.
2. **Validate** it with the Test Bench (transform parity) and the connectivity probe (every downstream reachable).
3. **Shadow it** through the tee relay in simulate mode and compare output to the incumbent until you are satisfied.
4. **Cut over** by repointing the upstream source at the MessageFoundry inbound and enabling real (non-simulated) egress for that feed's outbounds. Keep the incumbent's channel configured but idle.
5. **Watch** the feed in the browser web console the engine serves at `/ui` — per-message disposition, the delivery and audit trail, alerts, and the dead-letter queue with replay — for a soak period. Every message lands in exactly one of seven dispositions (received, routed, unrouted, filtered, processed, error, or not-deployed), so "where did that message go?" always has an answer.
6. **Roll back** if anything looks wrong: repoint the source back at the incumbent (or, during the shadow phase, simply stop the relay). Because the incumbent channel was left in place and your MessageFoundry change is a reviewed diff, rollback is fast and clean.

Because configuration lives in git and the database holds only messages and operational state, a feed's logic and its in-flight messages have independent blast radius: promoting new config never touches stored messages, and restoring the store never changes routing. Disaster recovery is two independent operations — redeploy config from git, restore data from a database backup.

Sequencing the waves

A pragmatic order:

1. **Start with low-risk, well-understood feeds** — a one-to-one ADT pass-through, or a file-based feed — to exercise your whole pipeline (author → check → test → shadow → cut over → monitor) end to end.
2. **Then consolidate the duplicated ones.** Feeds that share a transform or a destination across several incumbent channels are where the reuse-by-name model pays off — define the shared Handler or outbound once and reference it.
3. **Save the complex and the roadmap-dependent ones for last** — synchronous request/response feeds, WS-* mutual-TLS submissions, anything needing a payload format with no modeled lane yet, and anything that depends on a connector that does not ship.

Deployment guidance to plan for

A few operational points worth deciding before go-live, rather than discovering at cutover:

- **Bind interfaces deliberately.** Inbound MLLP/TCP listeners bind to a configured interface (loopback by default), overridable per connection. Exposing a listener off loopback is a per-environment operator decision, and the engine enforces it: a non-loopback plaintext listener is refused at start unless you explicitly override. A per-connection source IP allowlist can further restrict which peers may connect. Treat this as deployment configuration, not a limitation.
- **Egress is allowlisted, fail-closed.** Outbound hosts are gated by an allowlist — populate it for every downstream a migrated feed delivers to, or delivery is refused.
- **Pick your store for the target scale.** SQLite (zero-setup, single file) is the bundled default and suits pilots and single-node deployments; PostgreSQL or SQL Server is the production choice and is required for high availability.
- **Size against your partners, not a headline number.** The published throughput baseline is a record of what has actually been **measured**, on a named reference configuration, with the method published so you can re-run it on your own hardware — it is not a guarantee for your environment, and any figure presented as a *target* is not a demonstrated one. The dominant term is usually your partner's acknowledgement time, not the engine: a strictly-ordered feed sends, waits for the partner to acknowledge, then sends the next, so a 50 ms partner round-trip caps that one lane regardless of how fast the engine is. In the reference lab an ordered end-to-end lane runs on the order of 60 messages/second while intake (receive, persist, acknowledge) runs several times higher. Real capacity comes from **many interfaces**, not one giant pipe — split busy feeds at the source, and size for the peak hour rather than the daily average. Where one process is not enough, the built scaling axis is **engine sharding**: several engine processes partitioned by connection over one unified store.
- **High availability is active-passive.** Run identical engine processes against one shared server database; exactly one leader runs the graph, with warm standbys that take over on failure. Front it with a floating VIP or load balancer whose health check is a TCP connect to the listener port. Failover is a promotion window, not zero-downtime — prompt on a clean switchover, and bounded by the leadership lease on a crash (about 30 seconds with the shipped defaults, tunable) — which is another reason downstreams should be idempotent.

Security posture during and after migration

MessageFoundry carries PHI, so security is built in rather than bolted on, and a migration is a good moment to tighten it relative to the incumbent:

- **Authentication and RBAC** at a single API choke point, with deny-by-default per-route permissions and a tamper-evident audit log; every PHI access (raw view, replay) is recorded against the acting user.
- **Multi-factor authentication** for local accounts is on by default and covers every local account, with recovery codes; browser passkeys are available as an alternative second factor. Active Directory users' MFA stays delegated to your own identity provider.
- **Transport security is built and opt-in per surface**: TLS (and optional mutual TLS) on the engine's own API and WebSocket, and MLLP-over-TLS per connection, with outbound peer verification on by default. Turn it on for anything that leaves the box.
- **Interface authentication** options include mutual TLS, OAuth 2.0 client credentials on the HTTP-based outbounds, and SMART Backend Services for FHIR endpoints. Note the gap named earlier: the inbound HTTP listener has no on-socket request authentication yet, so put an exposed one behind a proxy that terminates auth.
- **Encryption at rest** — message bodies are encrypted in the store, with key rotation, on all three backends.
- **On-premises by default** — the engine API binds to loopback and requires authentication; no PHI leaves the local environment without explicit, reviewed configuration.

On assurance, we would rather be specific than flattering. A documented assessment against **OWASP ASVS 5.0 Level 3** exists and every control in it is either built or carries a written residual — but it is a **point-in-time, AI-assisted self-assessment, not a certification, not an audit, and not an independent review**, and we publish **no pass/fail count** while its scoring is being reconciled. There has been **no third-party assessment, no penetration test, and no dynamic testing** to date; that is a recorded, dated risk acceptance which is void on any production or off-loopback exposure, and an independent review is the right thing for an adopter to ask for. The assessment set itself is maintained privately and can be made available to evaluators under NDA — the rule deciding what is published and what is withheld is itself public, in the project's security-documentation policy.

On HIPAA, the accurate statement is narrow: MessageFoundry provides the technical safeguards the Security Rule expects of software in this role — access control, audit controls, integrity, and encryption in transit and at rest.

Compliance is a property of a covered entity's whole deployment and program, assessed by that entity and its counsel. No engine, this one included, confers it.

A short pre-flight checklist

- [] Every incumbent channel inventoried: source, destinations, message types, routing, transforms, ACK behavior, security, peak-hour volume, ordering requirement.
- [] Each channel decomposed into inbound Connection(s), Router(s), Handler(s), and outbound Connection(s).
- [] Every transport and payload format a feed depends on confirmed against the shipped connector list — anything missing scheduled into a later wave.
- [] Shared transforms and destinations identified and defined once, referenced by name.

- [] Config repo scaffolded (`messagefoundry init`); `environments/<env>.toml` set per environment; secrets supplied from the environment, never committed.
- [] `messagefoundry check` green; transforms confirmed with the Test Bench against synthetic messages.
- [] Connectivity probed to every downstream; egress allowlist populated; TLS configured for any off-loopback listener.
- [] Parallel run completed through the tee relay in simulate mode, output compared to the incumbent (test data only).
- [] Per-feed cutover plan with the incumbent channel left idle for fast rollback; not-yet-live partners committed as `deployed=false` .
- [] Monitoring, alerts, and dead-letter replay verified in the web console; soak period defined.

Further reading

- MessageFoundry connectors and settings:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/CONNECTIONS.md>
- The mental model — the four building blocks and how a message flows:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/MENTAL-MODEL.md>
- Understanding throughput, and how to size against your own systems:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/THROUGHPUT.md>
- The tee relay for parallel-run validation:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/TEE-RELAY.md>
- Security model and PHI handling:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/SECURITY.md>
- What security documentation is public, what is withheld, and how to ask:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/SECURITY-DOCS-POLICY.md>